

National University of Computer and Emerging Sciences



Lab Manual 08 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 8	4
Objectives:.....	4
Activities:	4
Activity 1 (Data Preprocessing in Python):.....	4
1. Why Data Preprocessing is Important.....	4
2. Missing Values	4
• Detecting Missing Values	4
• Handling Missing Values	5
○ Remove Missing Data	5
○ Imputation (Fill Values)	5
▪ Basic Statistical Imputation:	5
Mean (for numerical data).....	5
Median (robust to outliers)	5
Mode (for categorical data)	5
▪ Constant Value Imputation:	5
▪ Forward Fill	5
▪ Backward Fill	5
▪ Interpolation (for numerical sequences).....	5
▪ Group-Based Imputation	5
▪ Using Scikit-learn Imputers.....	5
Simple Imputer	5
KNN Imputation	5
Iterative Imputation	5
Which Method to Use?	6
3. Duplicate Data	6
4. Data Type Conversion	6
• Why Data Type Conversion is Important?	6
○ To Perform Calculations Correctly and To Avoid Run-Time Errors	6
○ To Enable Data Analysis	6
• Conversion	6
Options for error:	7
5. Feature Scaling	7
• What is Feature Scaling?	7
• Why is Feature Scaling Needed?	7

• Feature Scaling Types	7
○ Normalization	7
○ Standardization (Z-Score Scaling)	8
• Train-Test Feature Scaling.....	8
• When to Use Normalization vs Standardization	9
6. Encoding.....	10
• What is Encoding?	10
• Why is Encoding Needed?	10
• Examples of Encoding	10
• Types of Encoding	10
○ Label Encoding	10
○ One-Hot Encoding.....	11
○ Ordinal Encoding.....	11
• Encoding at Testing Time	12
Activity 2 (Problem Statements / Tasks):	12
1. Student Performance Data Cleaning	12
2. Sales Data Type Conversion & Validation	13
3. Feature Scaling Analysis	13
4. Encoding Categorical Data	13
5. Complete Preprocessing Pipeline	14
6. Train-Test Preprocessing Consistency.....	14

Lab Manual 8

Objectives:

- ✓ Handling Missing/Duplicate Values, Outliers
- ✓ Data type conversions
- ✓ Feature Scaling
- ✓ Encoding

Activities:

Activity 1 (Data Preprocessing in Python):

1. *Why Data Preprocessing is Important*

Data preprocessing is essential because:

- Raw data is often incomplete or inconsistent
- Machine learning models require clean numeric input
- Improves accuracy and model performance

2. *Missing Values*

Missing values occur when data is not available for some entries. This can happen due to data collection errors, incomplete surveys, or system failures. Missing data can introduce bias, lead to incorrect conclusions, and cause errors in computations.

City	State	Zip
Chicago	IL	<i>null</i>
San Antonio	TX	22259
<i>null</i>	TX	77343
<i>null</i>	TX	04927
San Diego	CA	16150
Philadelphia	PA	<i>null</i>
New York	<i>null</i>	25235
San Antonio	<i>null</i>	73868
Los Angeles	CA	24607
San Diego	CA	39289

- Detecting Missing Values

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

```
print(df.isnull().sum())    # Count missing values
```

```
print(df.info())           # Data types + missing info
```

- **Handling Missing Values**

- *Remove Missing Data*

Remove rows/columns with too many missing values:

```
df.dropna(inplace=True)          # Remove rows
df.dropna(axis=1, inplace=True)  # Remove columns
```

- *Imputation (Fill Values)*

- **Basic Statistical Imputation:**

Mean (for numerical data)

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

Median (robust to outliers)

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

Mode (for categorical data)

```
df['City'].fillna(df['City'].mode()[0], inplace=True)
```

- **Constant Value Imputation:**

Fill with a fixed value.

```
df['Age'].fillna(0, inplace=True)
df['City'].fillna("Unknown", inplace=True)
```

- **Forward Fill**

It uses previous value and useful for time-series data.

```
df.fillna(method='ffill', inplace=True)
```

- **Backward Fill**

```
df.fillna(method='bfill', inplace=True)
```

- **Interpolation (for numerical sequences)**

Works well for trends / continuous data

```
df['Age'] = df['Age'].interpolate()
```

- **Group-Based Imputation**

It fills based on groups and is more realistic in real datasets.

```
df['Age'] = df.groupby('City')['Age'].transform
                                     (lambda x: x.fillna(x.mean()))
```

- **Using Scikit-learn Imputers**

Simple Imputer

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
df[['Age']] = imputer.fit_transform(df[['Age']])
```

KNN Imputation

Uses similarity between rows

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=3)
df[['Age']] = imputer.fit_transform(df[['Age']])
```

Iterative Imputation

Predicts missing values using other features

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

```
imputer = IterativeImputer()
df[['Age']] = imputer.fit_transform(df[['Age']])
```

Which Method to Use?

Situation	Best Method
Numerical, no outliers	Mean
Numerical with outliers	Median
Categorical	Mode
Time-series	Forward/Backward fill
Pattern-based data	Interpolation
Real-world ML problems	KNN / Iterative

3. Duplicate Data

Duplicates can inflate dataset size and lead to incorrect insights.

```
print(df.duplicated().sum())    # Check duplicates
df = df.drop_duplicates()      # Remove duplicates
```

4. Data Type Conversion

- Why Data Type Conversion is Important?

Data type conversion is needed because data must be in the correct format to be used properly in analysis and machine learning.

- To Perform Calculations Correctly and To Avoid Run-Time Errors

If numeric data is stored as text (string), you cannot perform mathematical operations.
For Example:

```
price = "100"
print(price + price)    # Output: 100100
```

```
price = int("100")
print(price + price)    # Output: 200
```

```
"10" / 2    # Run-Time Error
int("10") / 2    # Works
```

- To Enable Data Analysis

Many operations require correct data types:

Operation	Required Type
Mean / Sum	Numeric
Sorting Dates Calculate duration Extract year/month	Datetime
Machine Learning	Numeric

- Conversion

```
# Convert to Numeric
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
# Convert to Date
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

```

# Convert Using astype()
# Used to explicitly convert a column to a specific data type
# (e.g., int, float, string). It is fast but raises errors if
# conversion fails and best used when data is already clean and
# consistent.
df['Quantity'] = df['Quantity'].astype(int)
# Smart Conversion - infer_objects()
# Attempts to convert columns of type object into more specific
# types like int, float, or datetime. It performs a soft
# conversion without forcing changes and useful when data types
# are not explicitly defined.
df = df.infer_objects()
# Auto Conversion - convert_dtypes()
# Automatically converts columns to the best possible data types
# (e.g., Int64, string, boolean) supported by pandas. It is more
# advanced than infer_objects() and preserves missing values
# properly and recommended for cleaning real-world datasets.
df = df.convert_dtypes()

```

Options for error:

Value	Description
'raise' (default)	Raise an error if conversion fails.
'coerce'	Invalid parsing will be set as NaN
'ignore'	Return original data if parsing fails.

5. Feature Scaling

- **What is Feature Scaling?**

Feature scaling is the process of standardizing or normalizing numerical features so that all values are on a similar scale. Feature scaling makes data comparable and improves model efficiency.

- **Why is Feature Scaling Needed?**

- It ensures that no feature dominates others due to larger values, improving model performance and accuracy.
- It is especially important for algorithms like KNN, SVM, and Gradient Descent, which depend on distance or magnitude.
- Tree-based models (e.g., Decision Trees, Random Forests) usually do not require feature scaling.

- **Feature Scaling Types**

- *Normalization*

Normalization rescales data to a fixed range, typically between 0 and 1. It preserves the shape of the original distribution but compresses the scale. Normalization is useful when:

- Data does not follow a normal distribution (skewness...)
- You are using distance-based algorithms (k-NN, k-means)
- Feature bounds are known and meaningful

Formula:

$$X_{\text{new}} = (X - X_{\text{min}})/(X_{\text{max}} - X_{\text{min}})$$

Python Code:

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Sample Data
X = np.array([[10], [20], [30], [40], [50]])

# Normalization
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)

print(X_norm)
```

- *Standardization (Z-Score Scaling)*

Standardization transforms data to have:

- Mean = 0
- Standard deviation = 1

It does not restrict values to a fixed range and works well when data follows a normal distribution.

Formula:

$$X_{\text{new}} = (X - \text{mean})/\text{Std}$$

Standardization is preferred when:

- Data contains outliers
- Features follow a Gaussian distribution
- Using algorithms like linear regression, logistic regression, SVM, or PCA

Python Code:

```
from sklearn.preprocessing import StandardScaler

# Sample data
X = np.array([[10], [20], [30], [40], [50]])

# Standardization
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized)
```

- **Train-Test Feature Scaling**

During testing (or prediction) time, feature scaling does not change its formula, but how it is applied changes.

- The scaler is fit only on the training data.
- During testing, the scaler is reused, not refit.
- The same parameters learned from training are applied to test data.

This ensures fairness and prevents data leakage.

Python Code:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np
```



```

# Sample dataset
X = np.array([[10], [20], [30], [40], [50]])

# Train-Test split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

# Fit scaler on training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Apply same scaler to test data
X_test_scaled = scaler.transform(X_test)

print("Train scaled:\n", X_train_scaled)
print("Test scaled:\n", X_test_scaled)

```

- **When to Use Normalization vs Standardization**

The choice between normalization and standardization depends mainly on the distribution of the data and the presence of skewness. If a feature is approximately normally distributed (symmetric and bell-shaped), standardization is preferred because it centers the data around zero and scales it based on variance. If the data is not normally distributed or has clear bounds, normalization is often more suitable, especially for distance-based algorithms.

Skewness is a key indicator in this decision. Skewness measures the asymmetry of a distribution and can be visually checked using a histogram or numerically computed using the `.skew()` function. A skewness value close to 0 indicates a symmetric distribution. In practice, if skewness lies roughly between -0.5 and $+0.5$, the data can be treated as approximately normal and standardization is recommended. If skewness is greater than $+1$ or less than -1 , the distribution is highly skewed, and normalization is generally preferred unless the data is transformed first (e.g., log transformation).

Python Code:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skew

# Sample data
data = np.array([10, 12, 15, 18, 20, 22, 25, 100])

# Histogram
plt.hist(data, bins=10)
plt.title("Histogram of Feature")
plt.show()

# Skewness value
skew_value = skew(data)
print("Skewness:", skew_value)

```

6. Encoding

- What is Encoding?

Encoding is the process of converting categorical (text) data into numerical form so that machine learning models can understand and process it.

- Why is Encoding Needed?

Most machine learning algorithms cannot work with text data, so encoding transforms categories into numbers. It ensures that models can learn patterns correctly without errors or misinterpretation.

- Examples of Encoding

- Label Encoding → Converts categories to numbers (e.g., Red=0, Blue=1)
- One-Hot Encoding → Creates binary columns for each category

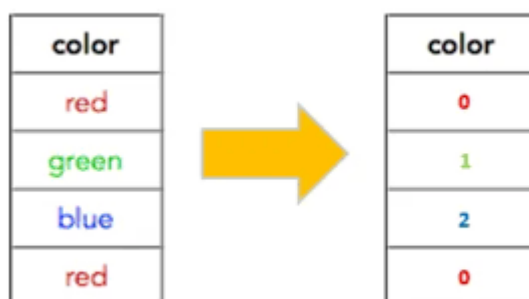
- Types of Encoding

- Label Encoding

Label Encoding converts each category into a unique numeric value. For example, categories like Red, Blue, and Green may be converted to 0, 1, and 2. It is simple and efficient but introduces an artificial order among categories. Therefore, it is suitable only when the categories have a natural order or for encoding target labels. It is commonly used for binary variables (Yes/No) or ordinal data.

Label encoding is suitable:

- When encoding the target variable (output label).
- When categories have a natural order (ordinal data) (with caution).
- When using tree-based models (Decision Tree, Random Forest), which are less sensitive to numeric ordering.



Python Code

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Categorical data
colors = ['Red', 'Blue', 'Green', 'Blue', 'Red']
df = pd.DataFrame({'Color': colors})

label_encoder = LabelEncoder()
df['Color_encoded'] = label_encoder.fit_transform(df['Color'])
print(df)
```

- *One-Hot Encoding*

One-Hot Encoding converts each category into a separate binary column. Each category is represented as 0 or 1, indicating absence or presence. For example, Color = Red, Blue becomes two columns: Red (1/0), Blue (1/0). It does not introduce any order, making it ideal for nominal (unordered) data. However, it can increase the number of columns when there are many categories.

One-Hot encoding is suitable when:

- When the feature is nominal (no natural order)
- When categories are independent (e.g., City, Color, Gender)
- When using distance-based models like KNN, SVM, Linear Regression
- When the number of categories is small to moderate



Color		Red	Yellow	Green
Red		1	0	0
Red		1	0	0
Yellow		0	1	0
Green		0	0	1
Yellow		0	0	1

Python Code:

```
import pandas as pd

countries = ['USA', 'Canada', 'India', 'USA', 'Canada']
df = pd.DataFrame({'Country': countries})

# One-Hot Encoding
one_hot = pd.get_dummies(df['Country'])
print(one_hot)
```

- *Ordinal Encoding*

Ordinal Encoding assigns numeric values to categories based on their meaningful order. For example, Small = 0, Medium = 1, Large = 2. It preserves the ranking information, which is important for certain features. Unlike Label Encoding, the order here is meaningful and intentional. It is commonly used for features like education level, size, or ratings.

Ordinal encoding is suitable when:

- When the feature has a clear, meaningful order
For Example: Low < Medium < High, Small < Medium < Large)
- When the ranking between categories is important for the model
- When you want to preserve relative importance between categories

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

Python Code:

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Ordinal data
severity = ['Low', 'Medium', 'High', 'Medium', 'Low']
df = pd.DataFrame({'Severity': severity})

encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
df['Severity_encoded'] = encoder.fit_transform(df[['Severity']])
print(df)
```

- **Encoding at Testing Time**

Encoding at testing time means applying the same encoding rules used during training. The encoder must be fitted only on training data and reused for test data. This ensures consistency between training and testing datasets. New or unseen categories should be handled carefully (e.g., ignored or assigned a default value). Incorrect encoding at test time can lead to errors or poor model performance.

Python Code:

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Training data
train = pd.Series(['Low', 'Medium', 'High'])

# Testing data
test = pd.Series(['Medium', 'Low'])
encoder = LabelEncoder()

# Fit on training data
encoder.fit_transform(train)

# Transform test data
test_encoded = encoder.transform(test)
print(test_encoded)
```

Activity 2 (Problem Statements / Tasks):

1. *Student Performance Data Cleaning*

Clean and preprocess the student dataset(csv file provided).

Requirements

- Detect missing values
- Handle missing values using:
 - Mean/Median (Age, Marks)
 - Mode (City)
- Convert:
 - Marks → numeric (handle "abc")
- Remove duplicate records

- Handle invalid values (e.g., negative age)

Display

- Cleaned dataset
- Summary of changes made

2. *Sales Data Type Conversion & Validation*

Prepare the dataset (csv file provided) for analysis.

Requirements

- Convert:
 - Price → numeric
 - Date → datetime
 - Quantity → integer
- Handle invalid values using errors='coerce'
- Identify rows with incorrect data
- Drop or fix invalid records

Display

- Data types before and after conversion
- Rows with invalid values

3. *Feature Scaling Analysis*

Apply and compare feature scaling techniques on housing related dataset (csv file provided).

Requirements

- Apply:
 - Min-Max Normalization
 - Standardization
- Plot histograms before and after scaling
- Calculate skewness

Display

- Original vs scaled values
- Skewness

Analysis

- Which scaling method is more appropriate and why?

4. *Encoding Categorical Data*

Encode categorical variables correctly (employees' dataset).

Requirements

- Apply:
 - Label Encoding → Gender
 - One-Hot Encoding → Department
 - Ordinal Encoding → Education (Bachelors < Masters < PhD)
- Split dataset into train/test
- Apply encoding properly (fit vs transform)

Display

- Encoded dataset

Analysis

- Why different encoding methods were used for each column

5. *Complete Preprocessing Pipeline*

Perform full data preprocessing (ecommerce dataset).

Requirements

- Handle missing values
- Remove duplicates
- Fix invalid values
- Convert data types
- Apply feature scaling on PurchaseAmount
- Encode City

Display

- Final processed dataset
- Step-by-step transformations

6. *Train-Test Preprocessing Consistency*

Demonstrate correct preprocessing workflow (Use any previous dataset).

Requirements

- Split data into train/test
- Apply:
 - Scaling
 - Encoding
- Ensure:
 - Fit only on training data
 - Transform both train and test

Display

- Train vs Test transformed values

Conceptual Question

- What happens if we fit scaler on test data? (Data Leakage)